



Time: 3 Hrs

Answer All Questions

Max Marks: 100

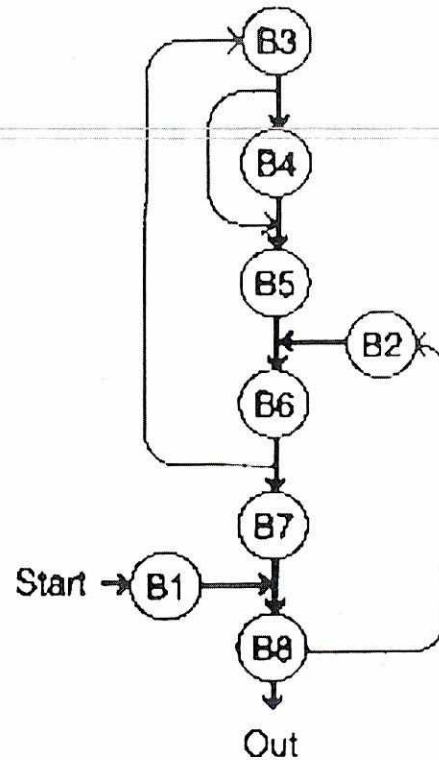
Note: All answers must be precise and to the point.

1.	a)	Explain the role of a lexer and suggest different ways of implementing a lexer for a language.	5
	b)	Diagrammatically explain the design of a lexical analyzer generated by the LEX tool with an example.	5
	c)	Explain in detail the working of front-end of a Compiler using the following input in C language : $\text{for}(i=0; i<10; i++) \text{ a}[i]=0;$	10
2.	a)	Briefly explain the model of Non-Recursive Descent Parser with a neat diagram.	5
	b)	Explain why the following grammar is not LL(k) for any k: $\begin{aligned} S &\rightarrow A \mid B \\ A &\rightarrow bbA \mid bb \\ B &\rightarrow bbB \mid b \end{aligned}$	5
	c)	Consider the following augmented grammar : $\begin{aligned} S' &\rightarrow S \\ S &\rightarrow L=L \mid id \\ L &\rightarrow L + id \mid id \end{aligned}$ The start state (Set of LR(0) items for this grammar is as follows: State 0 : $\begin{aligned} S' &\rightarrow \cdot S \\ S &\rightarrow \cdot L = L \\ S &\rightarrow \cdot id \\ L &\rightarrow \cdot L + id \\ L &\rightarrow \cdot id \end{aligned}$ Also we have, State 3 : $\begin{aligned} S &\rightarrow id \cdot \\ L &\rightarrow id \cdot \end{aligned}$ Because Follow(S) = { \$ } and Follow(L) = { \$, +, = } we will get a Reduce-Reduce conflict for State 3 in SLR parse table. Give the corresponding set of LR(1) items for State 0 and State 3, which are used to construct the canonical LR parser and explain why the conflict in SLR parser table is now eliminated.	5
	d)	Given the grammar: $\begin{aligned} S &\rightarrow A \\ A &\rightarrow A + A \mid B + + \\ B &\rightarrow a \end{aligned}$ Parse the string "a + + + a + +" using general style of Bottom-up parsing.	5

3.	a)	Given the grammar, write an SDD to generate Abstract Syntax tree : <pre> S -> S S epsilon S -> if (C) S S -> id = id ; C -> id rel id rel -> < > <= >= </pre>	10
		Justify your answer by carrying out the SDD over the input string : <pre> if(a > b) a = b; </pre>	
	b)	I. Write an L-attributed SDD for fetching the type of a basic declaration (example int a; or int a, b; etc) and update the information in symbol table. Convert your SDD into SDT scheme. II. Provide the implementation of the SDT scheme (from Part I) during predictive parsing.	10
4.	a)	Provide Quadruple representation of the following code : <pre> while (i ≥ n) i = i – a[i]; </pre>	5
	b)	Generate DAG for the following expression (assuming unambiguous expression grammar) : <pre> a + b * d - (a + b) * d / a + b * a + b </pre>	5
	c)	Convert the following program to SSA form: <pre> int main() { const int maxInput = 5; int i; double number, average, sum=0.0; for(i=1; i<=maxInput; ++i) { if(number < 0.0) goto jump; sum += number; } jump: average=sum/(i-1); printf("Sum = %f\n", sum); printf("Average = %f", average); return 0; } </pre>	5
	d)	Mention the criteria for code optimization. With an example explain the following terms : I. Induction variable II. Copy propagation	5

5. a) Perform any 5 optimizations on the following CFG for bubble sort program: Mention clearly the optimization performed and its result. Only variables are assumed to be live on exit.

B1	I := 1 GOTO B8
B2	J := 1 GOTO B6
B3	T1 := 4 * J T2 := A[T1] T3 := J + 1 T4 := 4 * T3 T5 := A[T4] IF T2 <= T5 GOTO B5
B4	T6 := 4 * J Temp := A[T6] T7 := J + 1 T8 := 4 * T7 T9 := A[T8] T10 := 4 * J A[T10] := T9 T11 := J + 1 T12 := 4 * T11 A[T12] := Temp
B5	J := J + 1
B6	IF J <= I GOTO B3
B7	I := I + 1
B8	IF I <= n - 1 GOTO B2



Pseudocode (For reference):

```

FOR I := 1 TO n - 1 DO
  FOR J := 1 TO I
    DO
      IF A[J] > A[J+1] THEN
        BEGIN
          Temp := A[J]
          A[J] := A[ J + 1 ]
          A[ J + 1 ] := Temp
        END
      END
    END
  END
END
  
```

- b) Generate target code for the following procedure assuming **stack allocation**: Stack area starts from address 500. Also, show the stack contents.

Procedure	Activation Record Size	Code Area
p()	20 bytes	100
q()	30 bytes	300

procedure p()

```

x = n * 2
param x
call q, 1
  
```

procedure q(i)

```

i = i * 2
return
  
```

- c) Write short notes on the issues in the design of a code generator.
- d) With a diagram explain how caller and callee might cooperate in managing the stack.